Tenarai Autonomous Enterprise Claim Review Agent

Groq GPT-OSS + LangGraph + Deterministic Rules + Human-in-the-Loop

Executive summary

Objective: automate claim-document review while keeping financial decisions auditable and human-governed. The LLM extracts facts and drafts controlled correspondence; deterministic Python services own validation and risk routing; LangGraph manages stateful orchestration and HITL pause/resume.

End-to-end workflow

Step	Purpose
1. Parse	Read pasted text, TXT or text-based PDF and normalize the claim packet.
2. Extract	Use Groq-hosted GPT-OSS with strict JSON schema to extract name, policy number, amount, conflicts and confidence.
3. Validate	Run deterministic Python checks: required fields, policy exists, claimant match, active policy and coverage limit.
4. Risk gate	Trigger HITL when amount exceeds $10,000, conflicting data appears, or confidence is low.
5. Human review	LangGraph interrupts and stores state until a caseworker approves/rejects with notes.
6. Draft letter	Generate formal approval/rejection correspondence for final sign-off.

Why this is enterprise-safe

- LLM does not decide claim eligibility; it only structures document facts and drafts language.
- Business rules are deterministic, testable and auditable.
- High-risk cases pause through a true LangGraph interrupt, not a UI-only warning.
- The workflow can be replayed through persisted state: extracted fields, validation results, escalation reason, reviewer decision and final letter.
- Provider abstraction allows Groq in the demo and Bedrock/Azure/OpenAI in production without redesigning the workflow.

Demo evidence

File	Scenario	Expected result
01_standard_low_value_approval.pdf	Jane Doe / POL-1001 / $4,250	Auto-approve, no HITL
04_high_value_hitl.pdf	Valid $18,750 claim	Policy passes, HITL required due to threshold
08_complex_multi_page_enterprise_exception.pdf	Multi-page packet with conflicting policies	Review + HITL exception summary

Interview Talking Points and CTO Narrative

Why LangGraph?

This is a stateful, long-running workflow with conditional routing and human interruption, not a chat-only application.

Why not let the LLM approve or reject?

Claim decisions have financial and regulatory consequences. The LLM extracts facts and drafts correspondence; deterministic services own eligibility.

How does HITL work?

If risk conditions are met, LangGraph interrupts the graph, checkpoints state using a thread ID, and resumes only after a caseworker submits approve/reject and notes.

How would this run in AWS?

S3 for claim upload, Textract for scanned PDFs, ECS/Fargate for the LangGraph worker, Aurora/Postgres for checkpoint and audit storage, Secrets Manager/KMS for secure configuration, and EventBridge/SNS for SLA escalation.

Where does RAG fit?

RAG is useful when eligibility depends on policy manuals or exclusion clauses. Retrieval supplies evidence; deterministic rules still govern final decisions.

What would you monitor?

Extraction accuracy, HITL rate, reviewer override rate, latency, cost per claim, failure rate and false approval/rejection rates.

3-minute Loom structure

Time	Message
0:00–0:30	Position the solution as a governed workflow: LLM extraction, deterministic decisions, LangGraph state and human accountability.
0:30–1:00	Run standard claim and show straight-through approval with generated draft letter.
1:00–2:00	Run high-value claim and show the HITL pause/resume path as the core demo moment.
2:00–2:30	Run complex packet and show conflict detection rather than model guessing.
2:30–3:00	Close with production AWS deployment and provider abstraction story.

Final executive line

AI interprets documents. Rules make policy decisions. Humans govern exceptions.